1. Experiment Title

Design and Implementation of a Lexical Analyzer (Manual Approach).

2. Objectives

After completing this experiment, students will be able to:

- Explain the function of a **lexical analyzer** in the compiler pipeline.
- Differentiate between **token, lexeme, and pattern**.
- Apply **regular expressions** to define token classes.
- Design and implement a **manual scanner**.
- Detect and report **lexical errors**.

3. Background Theory

3.1 Role of Lexical Analysis

Lexical analysis is the **first phase of compilation**, responsible for converting a stream of characters into a stream of tokens that will be used by the parser.

3.2 Important Definitions

- **Token:** A syntactic category (e.g., `IDENTIFIER`, `KEYWORD`)
- **Lexeme:** The actual string matched (e.g., `count`)
- **Pattern:** Rule describing tokens (typically via regular expressions)

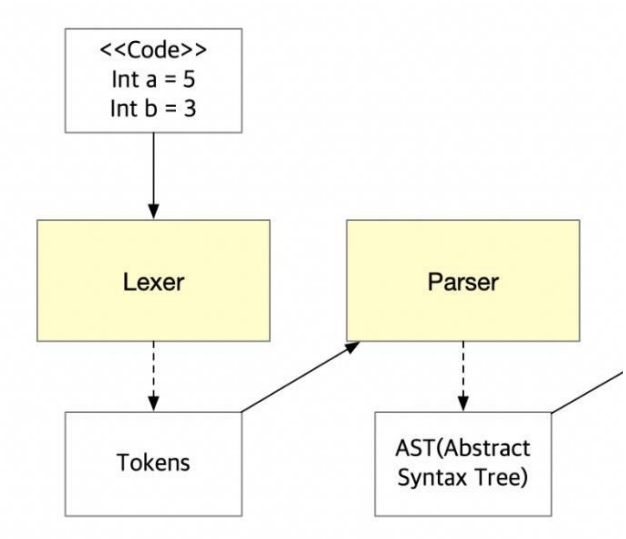
3.3 Token Categories

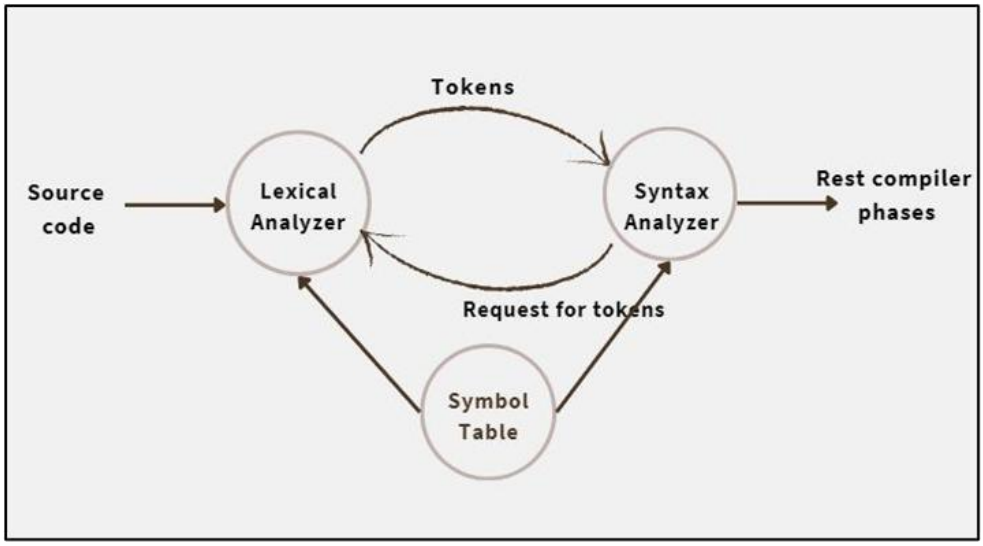
Token Type	Description	Examples
Keyword	Reserved words	`int`, `while`, `if`
Identifier	User-defined names	`x`, `sum`
Operator	Arithmetic/relational/logical ops	`+`, `-`, `=`
Literal	Constant values	`123`, `3.14`
Delimiter	Structural symbols	`;`, `,`, `()`

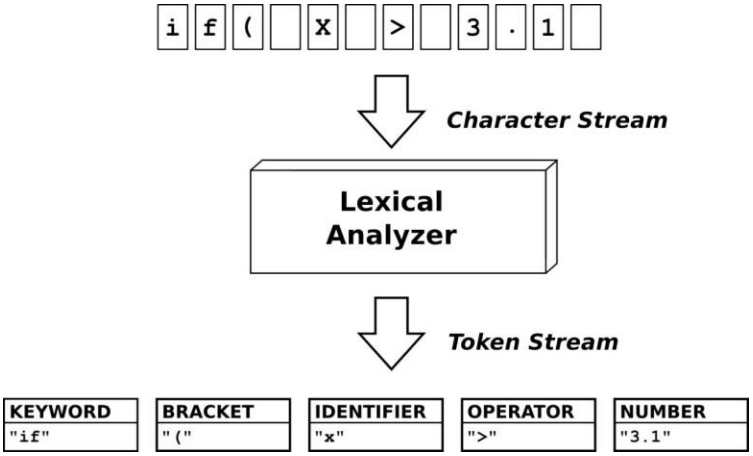
3.4 Regular Expressions for Token Specification

Token Type	Regular Expression
Identifier	`[a-zA-Z_][a-zA-Z0-9_]*`
Integer	`[0-9]+`
Float	`[0-9]+\.[0-9]+`
Operator	`[+\-*/=]`
Delimiter	`[;,(){}]`

3.5 Lexical Analysis Process







Process Overview:

1. Input source code is read as a stream of characters.
2. Characters are grouped into lexemes.
3. Lexemes are matched against patterns.
4. Tokens are generated.
5. Errors are reported if no valid pattern matches.

4. Problem Statement

Design a lexical analyzer that:

- Reads a source program from a file.
- Identifies tokens using **regular expressions (without using Lex/Flex)**.
- Outputs tokens in the format:

```
<TOKEN_TYPE, LEXEME>
```

5. Functional Requirements

Your program must:

- Recognize at least the following:
  - Keywords
  - Identifiers
  - Integer constants
  - Operators
  - Delimiters
- Ignore whitespace and newline characters.
- Distinguish between **keywords and identifiers**.
- Report invalid or unknown tokens.

6. Algorithm (Guideline)

Follow this structured approach:

1. **Input Handling**
   - Open and read the source file line by line.
2. **Preprocessing**
   - Remove or ignore whitespace.
   - Handle separators (space, newline, punctuation).
3. **Lexeme Formation**
   - Group characters into candidate lexemes.
4. **Pattern Matching**
   - Check lexeme against:
     - Keyword list
     - Identifier pattern
     - Numeric pattern
     - Operator set
     - Delimiter set
5. **Token Generation**

   - Output token in `<TYPE, LEXEME>` format.
6. **Error Handling**
   - If no pattern matches → mark as `INVALID`.
7. **Repeat**
   - Continue until end of file.

7. Design Constraints

- Do **not** use:
  - Lex / Flex
  - Yacc / Bison
- You **may use**:
  - Built-in regular expression libraries (if available)
  - Standard string processing functions
- Implementation language: **C / C++ / Java / Python**

8. Sample Input

```
int a = 10;
float b = a + 20;
```

9. Expected Output Format

```
<KEYWORD, int>
<IDENTIFIER, a>
<OPERATOR, =>
<NUMBER, 10>
<DELIMITER, ;>
...
```

(Exact output may vary depending on implementation choices, but format must be consistent.)

10. Test Cases (Minimum Requirement)

Students must test with:

Basic Case

- Simple declarations and assignments

Intermediate Case

- Multiple statements
- Mixed tokens

Edge Case

- Invalid identifiers (e.g., `1abc`)
- Unknown symbols (e.g., `@`, `#`)

11. Post-Lab Extensions (Optional)

Students can enhance their program to:

- Support floating-point numbers
- Recognize comments (`//`, `/* */`)
- Handle string literals
- Track line numbers for errors

12. Viva Questions (sample)

- What is the difference between token and lexeme?
- Why are regular expressions used in lexical analysis?
- What is the longest match rule?
- How does a lexical analyzer interact with a parser?
- What are lexical errors?

13. Submission Requirements

Students prepare and submit single document. Main body of the document contains the source code of the program. Other information needs to be added as appendices in the document.

- Source code
- Sample input and output
- Brief explanation of:
    - Token definitions
    - Approach used